

Build with Snowpark Using Java, Scala, and Python

Alan Biju Palayil

Department of Information Technology, University of the Cumberlands

MSDS-535: Data Mining

Professor: Dr. Kathryn Wifvat

Date: June 7th, 2026

Building with Snowpark Using Java, Scala, and Python

Introduction

As organizations continue expanding their use of cloud analytics and machine learning, the need for scalable data engineering platforms has become increasingly important. Traditional data pipelines often require moving data between storage platforms, processing engines, and machine learning environments. This creates additional complexity, cost, and security concerns. Snowflake addresses many of these challenges through Snowpark, a developer framework that allows data transformation, analytics, and machine learning workloads to be built directly inside the Snowflake platform.

Snowpark extends Snowflake beyond traditional SQL-based analytics by allowing developers to use familiar programming languages such as Java, Scala, and Python while keeping computation close to the data. This creates a modern cloud-native development environment where applications, transformations, and machine learning workflows can run directly inside Snowflake warehouses.

This report summarizes how Snowpark can be built using Java, Scala, and Python, discusses its advantages for enterprise analytics, and reflects on how it can be used to scale modern data operations.

What Snowpark Is

Snowpark is Snowflake's developer framework for building data applications and engineering workflows inside Snowflake. It provides DataFrame APIs and programming support beyond SQL, allowing developers to interact with Snowflake data using familiar coding languages instead of relying entirely on traditional SQL queries.

A major advantage of Snowpark is that processing happens inside Snowflake's compute layer, which means large datasets do not need to be exported into separate systems for processing. This reduces unnecessary data movement while improving performance, governance, and security.

Snowpark enables teams to:

- build ETL and ELT pipelines
- transform structured and semi-structured data
- create machine learning workflows
- develop predictive analytics applications
- integrate AI and advanced analytics with Snowflake data

This makes Snowpark an important platform for modern cloud-native analytics and enterprise data engineering.

Building with Python in Snowpark

Python is one of the most powerful and practical languages supported in Snowpark. Because Python is widely used in machine learning, data science, automation, and analytics, Snowpark's Python support makes it highly accessible to developers and analysts.

Using Snowpark Python APIs, developers can:

- create DataFrames
- read Snowflake tables
- transform data
- filter and aggregate records
- join datasets
- build feature pipelines
- run machine learning workflows

Snowpark Python integrates well with libraries commonly used in data science, including:

- pandas
- NumPy
- scikit-learn
- machine learning model frameworks

This allows organizations to build predictive models while keeping enterprise data inside Snowflake rather than exporting data to external notebooks or compute environments.

For teams already working with Python-based machine learning or analytics workflows, Snowpark makes adoption relatively straightforward.

Building with Java

Java support in Snowpark makes it attractive for enterprise application development and large-scale production systems. Many enterprise environments already rely heavily on Java for backend systems, APIs, and internal application architecture.

Using Snowpark with Java allows developers to:

- build scalable data applications
- process Snowflake data programmatically
- integrate with enterprise systems

- automate workflows using object-oriented code
- build reusable business logic inside Snowflake

Java is especially useful when integrating Snowflake into existing enterprise software environments because many organizations already maintain Java-based platforms in production.

This makes Snowpark practical not only for analytics teams, but also for engineering teams building enterprise-scale applications.

Building with Scala

Scala support is particularly valuable for organizations already working with Apache Spark or distributed big data systems. Many data engineers are already familiar with Scala-based DataFrame operations, so Snowpark feels similar to Spark development.

Using Scala with Snowpark enables:

- distributed transformations
- scalable data engineering pipelines
- structured data manipulation
- advanced data processing logic
- easier migration from Spark-based workflows

Because Scala aligns closely with modern data engineering practices, it provides strong flexibility for organizations already managing large-scale cloud pipelines.

Snowpark's similarity to Spark DataFrame concepts reduces the learning curve for developers transitioning into Snowflake environments.

Benefits of Snowpark for Enterprise Operations

Snowpark offers several important operational advantages for enterprise analytics environments.

1. Reduced Data Movement

Traditional pipelines often require exporting data to external tools before transformation or model development. Snowpark allows processing directly inside Snowflake, minimizing movement between systems.

2. Improved Performance

Because compute occurs inside Snowflake warehouses, Snowpark benefits from Snowflake's scalable cloud infrastructure and elastic compute performance.

3. Better Security and Governance

Keeping data inside Snowflake improves governance, reduces unnecessary exposure, and simplifies access control management.

4. Multi-Language Flexibility

Support for Python, Java, and Scala allows organizations to build using tools already familiar to their teams.

5. Strong Machine Learning Integration

Snowpark supports machine learning workflows while integrating naturally with enterprise analytics platforms and modern data science processes.

According to Han et al. (2022), modern analytics systems require scalable infrastructure capable of supporting both data processing and advanced modeling. Snowpark aligns strongly with this requirement by combining compute, analytics, and development inside a single cloud-native platform.

Practical Enterprise Use Cases

Snowpark can be applied across many industries and use cases.

Examples include:

- customer segmentation and analytics
- fraud detection models
- financial reporting automation
- predictive forecasting
- ETL pipeline orchestration
- recommendation systems
- machine learning feature engineering
- operational dashboards
- AI-assisted analytics workflows

In financial systems or enterprise analytics environments, Snowpark can be especially useful because it combines scalable data processing with governance and centralized cloud storage.

Conclusion

Snowpark expands Snowflake from a cloud data warehouse into a full developer and analytics platform. By supporting Java, Scala, and Python, Snowpark gives organizations

flexibility to build applications, transformations, and machine learning workflows using the languages and tools their teams already know.

Among the supported languages, Python is particularly powerful because of its role in machine learning and analytics, while Java and Scala make Snowpark practical for enterprise software engineering and large-scale data engineering.

The ability to run code directly inside Snowflake reduces data movement, improves security, increases performance, and simplifies operational management. For organizations building modern cloud-native analytics platforms, Snowpark provides a scalable and highly flexible development framework.

Overall, Snowpark represents an important evolution in enterprise data engineering and is a valuable platform for organizations looking to scale analytics, machine learning, and data operations efficiently.

References

Aggarwal, C. C. (2015). *Data mining: The textbook*. Springer.

Han, J., Pei, J., & Kamber, M. (2022). *Data mining: Concepts and techniques* (4th ed.). Elsevier.

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.